H

Why I picked Claude Code (and what each of the others is good at)

> **Note**
>
> *As of 2026-05. The AI-tool landscape evolves faster than print runs do, so treat the specific products below as a snapshot: the way to read it is for the lasting logic (what each kind of tool is shaped for), which changes far more slowly than the feature lists.*
>
> *You finished the book. You built a staff. Fair question: why this tool and not one of the others people keep telling you about? Here is the honest answer, including the cases where one of the others is the better pick.*

The wedge (one paragraph)

The reason this book is about Claude Code and not one of the others is the conductor's notation rule. Every chapter teaches you to *prompt* Claude rather than *write code*, and the runtime underneath that promise is unusually deep. The `.claude/` folder you built across twenty-nine chapters is a portable portfolio: a memory file, a commands directory, agents that act as staff, skills that act as procedures, hooks that fire on events, MCPs that wrap services, and scheduled jobs that run while you make coffee. None of the alternatives ship that whole shape in one place. They each ship a piece of it, beautifully (an editor, an agent runtime, a cross-device automator, a UI scaffolder) and what you trade for the depth in any one piece is the *integration* of all of them. If your goal is *to build an AI staff that runs without you, while staying inside English as your only programming language*, Claude Code is the tool that was designed for that exact sentence. The others were designed for different sentences. The next pages tell you which.

The four categories

1. Terminal-native agentic AI: *"Claude lives in your shell"*

This is Claude Code's category. The category isn't crowded yet, but it has real entrants. *[as of 2026-05]*

Claude Code (the book's choice). *Best at:* cross-system orchestration, file work, MCPs, hooks, scheduling, multi-agent. *Worst at:* GUI-heavy work (GUI = the point-and-click visual interface, as opposed to typing

commands) that needs a designer's eye; raw inline-completion speed (inline completion = the editor finishing your code as you type, like autocomplete for programmers) inside source code. *Why I picked it:* the breadth of the agentic surface (files, shell, browser, computer, schedule, agents) is the broadest in the category. The MCP ecosystem is large and growing; the plugin and skills marketplaces (Appendix C) are filling out fast.

Codex CLI (OpenAI) (CLI = command-line interface, a tool you run by typing in the terminal rather than clicking). *Best at:* code generation in pure-coding contexts; OpenAI-ecosystem tooling. *Worst at:* the cross-system glue Claude Code is built for. *When to reach for it:* if your work is 90% inside source code and you want to live in OpenAI's stack, Codex CLI's center of gravity is the editor/repo, not the `.claude/`folder.

Aider. *Best at:* bring-your-own-model open source workflow. *Worst at:* multi-agent orchestration. *When to reach for it:* if you have a strong opinion about which model you want to drive, and you want a tool that gets out of the way between you and that model.

2. IDE-embedded AI: *"AI inside your editor"*

(IDE = integrated development environment, the all-in-one code editor app developers work in.)

This category is mature, well-funded, and where most working developers live. *[as of 2026-05*]

Cursor. *Best at:* tab-completion (press *Tab* and the editor fills in the next chunk of code), multi-file edits, inline refactor. *Worst at:* unattended work and scheduled cross-system orchestration. *When to reach for it:* if your work is *"I'm writing code and I want AI to help me write it faster"* and not *"AI does the job, I supervise."*

GitHub Copilot. *Best at:* tight Microsoft/GitHub integration; well-supported in JetBrains and VS Code. *Worst at:* agentic surface beyond the editor. *When to reach for it:* if your team is already in GitHub Enterprise and centralized billing matters.

Windsurf. *Best at:* a more agentic IDE experience than Cursor. *Worst at:* pure terminal workflows. *When to reach for it:* if you want a halfway point between IDE and Claude Code, agent runs that supervise themselves a little more than Cursor does, but still inside an editor surface.

3. Chat surface AI: *"the chat window"*

The category that ChatGPT defined. Different shape from the rest of this appendix. *[as of 2026-05*]

Claude.ai (Desktop / Web). *Best at:* one-off creative thinking, brainstorming, drafting, deep document analysis. *Worst at:* anything you want to repeat or schedule. *When to reach for it:* when you're thinking, not building. The brain says *"once,"* not *"every Monday."*

ChatGPT (Web / Desktop / Mobile). *Best at:* fast iterative thinking, image generation, voice mode, broadest model menu. *Worst at:* persistent cross-system state; file-system access on your machine. *When to reach for it:* same use cases as `Claude.ai` with different model preferences.

Perplexity. *Best at:* live-web research with citations. *Worst at:* anything that requires writing files or running code. *When to reach for it:* when you need the answer fast, with sources.

4. Builder / agent platforms: *"build agents in a UI"*

The category that's growing fastest. By the time you read this there will be more entrants. *[as of 2026-05*]

Lovable / v0 / Bolt. *Best at:* shipping a web app fast from a single prompt with a hosted preview. *Worst at:* working against an existing codebase you control. *When to reach for it:* zero-to-one website or app, especially for non-coders. *Chapter 22*'s website chapter discusses where Lovable would have been faster and where Claude Code's repo-aware approach won.

OpenClaw / Claude Cowork. *Best at:* spawning parallel Claude sessions for independent tasks; the multi-agent dispatch pattern. *Worst at:* anything that needs a human in the loop on every step. *When to reach for it:* once you've earned multi-agent fluency in Chapters 18–20 and want to push further into parallel work.

Replit Agent / Antigravity. *Best at:* end-to-end app development with their own runtime. *Worst at:* working with your existing tools. *When to reach for it:* greenfield app dev (greenfield = a brand-new project with nothing built yet) where vendor lock-in (getting stuck on one company's platform because moving off it is painful) is acceptable.

The "if I had to pick again" admission

The point of this section is that the recommendation isn't *"Claude Code is best for everyone."* It's *"Claude Code is best for the operator pattern this book teaches, and here are the four cases where another tool would actually beat it."*

Case 1: ***If your work is 95% writing code in an existing codebase.*** Cursor (or Windsurf) would be a defensible primary. Claude Code becomes a secondary for the cross-system jobs that crop up around the editing work. The editor surface is where your day already lives; an editor-native tool serves you better than a terminal-native one.

Case 2: ***If you don't have a desktop computer and are exclusively on mobile.*** The chat surfaces win by default; Claude Code can't be your primary. Claude.ai and ChatGPT are the options. The book's promise, a small staff that runs unattended on a machine you own, assumes you have a machine you own.

Case 3: ***If you're building a single greenfield app and have no other systems.*** Lovable or v0 will be faster to first deploy than Claude Code. They were built for that exact sentence. Claude Code's strength is integration *across* systems; if you have one system to build and no others to integrate with, you're using the broader tool when a narrower one would ship sooner.

Case 4: ***If you're already deeply in the OpenAI ecosystem (GPT, Sora, Codex).*** The integration story is real and Codex CLI may be a better primary than Claude Code. The same shape of book could be written about Codex; it would be a different book with a different staff, but the operator pattern translates.

The reader who knows when to leave the recommendation is a reader who can actually *use* the recommendation. That is the whole point of writing the appendix this way.

Reference panel: quick-reach lookup

If you want to...	Reach for...
Draft a memo or think out loud	`Claude.ai` or ChatGPT (chat)

If you want to...	Reach for...
Cross-system job (read files, call APIs, the connectors apps use to talk to each other, drive browser)	Claude Code
Tab-complete inside source code	Cursor or Copilot
Ship a website with no codebase	Lovable / v0
Live-web research with citations	Perplexity
Spawn parallel agents in isolation	OpenClaw / Cowork (post-*Chapter 20*)
Audit your shell history with AI	Claude Code (*Chapter 25*)
Run a job nightly without you	Claude Code (*Chapter 9* + *Chapter 24*)

Evaluating any new entrant

The shape of each tool, the wedge it optimizes around, changes much more slowly than feature lists. Take any new entrant, ask: *what surface is it for, what artifact does it produce, who is the operator, what's the integration story for the other pieces of the job?* Those four questions outlast any product on this page; run a new tool through them and you can place it yourself without waiting for someone else to update a list.

Claude Code is not best at everything. It is best at *the specific thing this book's reader wanted to do*: build a small AI staff, conduct it in English, keep the whole portfolio in one folder, stop being the bottleneck for your own week. If a different sentence describes you better, the alternatives above are good. If the sentence still describes you, you already know where to go. Open the terminal. Type `claude`. The staff is upstairs.